Guía del Desarrollador: Recomendador con Gemini Embeddings

Estructura de datos, ChromaDB, personalización semántica y lógica multiusuario

Esta guía detalla la implementación técnica para construir un sistema de recomendación híbrido basado en embeddings vectoriales empleando la API de Gemini y una base de datos local como ChromaDB. Se cubren los aspectos de recolección, formateo de texto avanzado y emparejamiento de múltiples perfiles.

1. Instalación del Entorno y Dependencias

Para levantar este proyecto en local, necesitas instalar las librerías de persistencia vectorial, el SDK de Google GenAI y herramientas de peticiones HTTP en tu script de Python:

```
pip install chromadb google-genai requests
```

2. Estructura del Documento JSON de Películas

Antes de generar embeddings, necesitas consolidar la información en un archivo local (por ejemplo, `peliculas.json`). Cada película debe contar con datos separados para el texto explicativo y metadatos estructurados de filtrado:

```
{
  "id_tmdb": 27205,
  "titulo": "Inception",
  "director": "Christopher Nolan",
  "generos": ["Ciencia Ficción", "Thriller", "Acción"],
  "voto_promedio": 8.4,
  "total_votos": 14200,
  "sinopsis": "Un ladrón con la habilidad de entrar en los sueños de los demás roba
secretos corporativos...",
  "reviews_populares": [
    "Una genialidad visual que desafía la lógica y te mantiene al borde del asiento.",
    "El diseño de producción y la banda sonora de Zimmer son perfectos, un clásico
moderno."
  ]
}
```

3. El Secreto del "Peso" y la Personalización del Texto

> **¡Atención Técnica Importante!** No agregues campos como `"Importancia: 9/10"` dentro del string que envías al embedder. Los modelos de embedding miden **similitud semántica (significado)**, no

> álgebra matemática. Si pones "Importancia: 10", el sistema agrupará películas que digan "Importancia 10" entre sí, rompiendo la lógica del contenido.

Para solucionar el problema de penalizar o premiar películas basándote en la popularidad (ej. una nota de 10 con pocos votos frente a un 8 con millones de votos), tienes dos estrategias profesionales:

Estrategia A: Enriquecimiento Semántico Natural (Para el Embedder)

Traduce los números fríos a prosa que el modelo de lenguaje comprenda perfectamente. Reestructura el generador de texto de tu ficha para que use adjetivos según rangos de popularidad:

```
# Lógica en Python para transformar popularidad en lenguaje natural
if total_votos > 500000:
    impacto_social = "Esta es una película de culto masivo a nivel mundial, vista y
aclamada por millones de espectadores."
elif total_votos < 2000:
    impacto_social = "Esta es una obra oculta de nicho, una joya cinematográfica
independiente con muy poca difusión comercial."
else:
    impacto_social = "Una película con una recepción estable dentro del circuito comercial
general."
```

Estrategia B: Reranking Híbrido post-búsqueda (Recomendado)

Deja que el embedder solo se encargue de buscar las "vibras" y temáticas de las películas. Guarda los votos y puntuaciones en los **metadatos** de ChromaDB. Cuando hagas la consulta, recupera los 50 resultados más parecidos y vuelve a ordenarlos matemáticamente en Python usando una fórmula de votación Bayesiana o un multiplicador:

```
score_final = score_similitud_vectorial * (voto_promedio * log(total_votos))
```

4. Implementación en Python (Script Completo)

A continuación, se presenta el código modular para construir la base de datos, insertar películas con metadatos y realizar búsquedas personalizadas.

```
import json
from google import genai
from google.genai import types
import chromadb

# 1. Inicializar Clientes (Asegúrate de tener la variable GEMINI_API_KEY en tu entorno)
client = genai.Client()
chroma_client = chromadb.PersistentClient(path="./mi_base_datos_vectorial")

# Crear o cargar la colección
coleccion = chroma_client.get_or_create_collection(name="peliculas_recomendador")

def generar_texto_enriquecido(peli):
```

```
    # Construcción semántica inteligente
    popularidad_texto = "Aclamada masivamente por millones de personas." if
peli["total_votos"] > 100000 else "Una película selecta de nicho independiente."

    texto_ficha = (
        f"Título: {peli['titulo']}. Director: {peli['director']}. "
        f"Géneros: {', '.join(peli['generos'])}. Sinopsis: {peli['sinopsis']} "
        f"Contexto de recepción: {popularidad_texto} "
        f"Comentarios del público: {' '.join(peli['reviews_populares'])}"
    )
    return texto_ficha

def indexar_peliculas(lista_peliculas):
    for peli in lista_peliculas:
        texto_para_embedding = generar_texto_enriquecido(peli)

        # Invocar API de Gemini para obtener el vector
        respuesta = client.models.embed_content(
            model="text-embedding-004",
            contents=texto_para_embedding
        )
        vector = respuesta.embeddings[0].values

        # Insertar en ChromaDB (Separando Texto, Vector y Filtros)
        coleccion.add(
            ids=[str(peli["id_tmdb"])],
            embeddings=[vector],
            documents=[texto_para_embedding],
            metadatas=[{
                "titulo": peli["titulo"],
                "generos": ",".join(peli["generos"]),
                "voto_promedio": peli["voto_promedio"],
                "total_votos": peli["total_votos"]
            }]
        )
    print("¡Indexación completada con éxito!")

# Ejemplo de Consulta para Combinar Perfiles de 2 Personas
def buscar_peli_ideal_grupo(genero_filtro, ids_vistas_usuarios, peli_referencia_txt):
    # Generar vector para la búsqueda (Ej: "Película parecida a Interestelar")
    res_query = client.models.embed_content(model="text-embedding-004",
contents=peli_referencia_txt)
    vector_query = res_query.embeddings[0].values

    # Realizar consulta aplicando filtros duros de metadatos de ChromaDB
    resultados = coleccion.query(
        query_embeddings=[vector_query],
        n_results=20,
        where={"generos": {"$contains": genero_filtro}}
    )

    # Filtrado en memoria de películas ya vistas por cualquiera del grupo
    peliculas_filtradas = []
```

```
    for i in range(len(resultados['ids'][0])):
        peli_id = resultados['ids'][0][i]
        peli_metadata = resultados['metadatas'][0][i]

        if peli_id not in ids_vistas_usuarios:
            peliculas_filtradas.append(peli_metadata["titulo"])

    return peliculas_filtradas
```

5. Adaptación al ecosistema de Spotify

El mismo concepto aplica de forma magnífica para canciones utilizando la API de desarrolladores de Spotify:

Elemento Cinematográfico	Equivalente en Spotify	Datos para el Texto / Embedder
Sinopsis de la trama	Audio Features / Letras	"Canción de alta energía, ritmo bailable acelerado, acústica baja."
Director de la película	Artista o Productor	"Producido por Bizarrap, interpretado por Quevedo."
Géneros cinematográficos	Playlists donde aparece	"Asociada a microgéneros: Urbano Latino, Trap Argentino."
Reseñas de Letterboxd	Comentarios o Tags	Descripción de la vibra de la canción en redes o listas de éxitos.

Almacenando las canciones con esta estructura semántica en ChromaDB, podrás cruzar los perfiles de reproducción de dos personas para generar la *playlist perfecta para un viaje en coche* excluyendo las canciones quemadas o saltadas frecuentemente.